

Operating Systems Lab Assignment 3 -

Task1- Implement priority and round robin scheduling and calculate turn around time and waiting time.

Sol.- Priority scheduling

```
(kashvi@LAPTOP-2SJNMAE1)-[~]  
$ nano task1_priority.py
```

```
kashvi@LAPTOP-2SJNMAE1: ~$ nano task1_priority.py  
n = int(input("Enter no. of processes: "))  
bt=[]; at=[]; pr=[]  
  
for i in range(n):  
    b,a,p = map(int,input(f"P{i+1} (burst arrival priority): ").split())  
    bt.append(b); at.append(a); pr.append(p)  
  
ct=[0]*n; tat=[0]*n; wt=[0]*n; done=[0]*n; t=0  
  
for _ in range(n):  
    avail = [i for i in range(n) if not done[i] and at[i]<=t]  
    if not avail: t=min(at[i] for i in range(n) if not done[i]); continue  
    i = min(avail,key=lambda x:pr[x]); t+=bt[i]; ct[i]=t; done[i]=1  
  
for i in range(n): tat[i]=ct[i]-at[i]; wt[i]=tat[i]-bt[i]  
  
print("P\tBT\tAT\tPR\tCT\tTAT\tWT")  
  
for i in range(n): print(f"P{i+1}\t{bt[i]}\t{at[i]}\t{pr[i]}\t{ct[i]}\t{tat[i]}\t{wt[i]}")  
  
print("Avg TAT=",round(sum(tat)/n , 2) , " Avg WT=",round(sum(wt)/n , 2))
```

```
(kashvi@LAPTOP-2SJNMAE1)-[~]
$ python3 task1_priority.py
Enter no. of processes: 3
P1 (burst arrival priority): 5 0 2
P2 (burst arrival priority): 3 1 1
P3 (burst arrival priority): 8 2 3
P      BT      AT      PR      CT      TAT      WT
P1      5      0      2      5      5      0
P2      3      1      1      8      7      4
P3      8      2      3     16     14      6
Avg TAT= 8.67 Avg WT= 3.33
```

➔ Round Robin Scheduling

```
(kashvi@LAPTOP-2SJNMAE1)-[~]
$ nano task1_roundrobin.py
```

```
kashvi@LAPTOP-2SJNMAE1: ~ × + v
GNU nano 8.1 task1_roundrobin.py *
n = int(input("n: "))
bt,at=[],[]

for i in range(n):b,a=map(int,input().split());bt+=b;at+=a
q = int(input("q: "))
r = bt[:];t=0;ct=[0]*n;Q=[]

while 1:
    for i in range(n):
        if at[i]<=t and r[i]>0 and i not in Q:Q+=i
    if not Q:
        if all(x==0 for x in r):break
        t+=1;continue
    i=Q.pop(0);x=min(q,r[i]);r[i]-=x;t+=x
    for j in range(n):
        if at[j]<=t and r[j]>0 and j not in Q:Q+=j
    if r[i]>0:Q+=i
    else:ct[i]=t

tat = [ct[i]-at[i]for i in range(n)]
wt = [tat[i]-bt[i]for i in range(n)]
print("P\tCT\tTAT\tWT")

for i in range(n):print(f"{i+1}\t{ct[i]}\t{tat[i]}\t{wt[i]}")
print("Avg TAT=",round(sum(tat)/n,2)," Avg WT=",round(sum(wt)/n,2))
```

```
(kashvi@LAPTOP-2SJNMAE1)-[~]  
$ python3 task1_roundrobin.py
```

```
n: 3
```

```
5 0
```

```
4 1
```

```
2 2
```

```
q: 2
```

P	CT	TAT	WT
---	----	-----	----

1	9	9	4
---	---	---	---

2	11	10	6
---	----	----	---

3	8	6	4
---	---	---	---

Avg TAT= 8.33 Avg WT= 4.67

Task 2- Simulate worst-fit, best-fit, first-fit memory allocation strategies.

Sol.-

```
(kashvi@LAPTOP-2SJNMAE1)-[~]  
$ nano task2.py
```

```
GNU nano 8.1 task2.py *  
def allocate(blocks, procs, strat):  
    b_used=[False]*len(blocks)  
    for i,p in enumerate(procs):  
        idx=None  
        if strat=='first':  
            for j,b in enumerate(blocks):  
                if not b_used[j] and b>=p: idx=j; break  
        elif strat=='best':  
            idx=min((j for j in range(len(blocks)) if not b_used[j] and blocks[j]>=p),key=lambda j:blocks[j],default=None)  
        else:  
            idx=max((j for j in range(len(blocks)) if not b_used[j] and blocks[j]>=p),key=lambda j:blocks[j],default=None)  
        if idx is not None:  
            b_used[idx]=True  
            print(f"P{i+1}({p}) → Block{idx+1} ({blocks[idx]-p} internal frag)")  
        else:  
            print(f"P{i+1}({p}) → Not Allocated")  
  
blocks=list(map(int,input("Enter memory block sizes: ").split()))  
  
procs=list(map(int,input("Enter process sizes: ").split()))  
  
s=input("Strategy (first/best/worst): ").lower()  
  
allocate(blocks,procs,s)  
|
```

```
(kashvi@LAPTOP-2SJNMAE1)~  
$ python3 task2.py  
Enter memory block sizes: 100 500 200 300 600  
Enter process sizes: 212 417 112 426  
Strategy (first/best/worst): first  
P1(212) → Block2 (288 internal frag)  
P2(417) → Block5 (183 internal frag)  
P3(112) → Block3 (88 internal frag)  
P4(426) → Not Allocated
```

```
(kashvi@LAPTOP-2SJNMAE1)~  
$ python3 task2.py  
Enter memory block sizes: 100 500 200 300 600  
Enter process sizes: 212 417 112 426  
Strategy (first/best/worst): best  
P1(212) → Block4 (88 internal frag)  
P2(417) → Block2 (83 internal frag)  
P3(112) → Block3 (88 internal frag)  
P4(426) → Block5 (174 internal frag)
```

```
(kashvi@LAPTOP-2SJNMAE1)~  
$ python3 task2.py  
Enter memory block sizes: 100 500 200 300 600  
Enter process sizes: 212 417 112 426  
Strategy (first/best/worst): worst  
P1(212) → Block5 (388 internal frag)  
P2(417) → Block2 (83 internal frag)  
P3(112) → Block4 (188 internal frag)  
P4(426) → Not Allocated
```

Task 3 - Implement MFT and MVT Memory Management

Sol.- MFT (fixed partition)

```
(kashvi@LAPTOP-2SJNMAE1)-[~]  
$ nano task3_mft.py
```

```
kashvi@LAPTOP-2SJNMAE1: ~  
GNU nano 8.1 task3_mft.py *  
parts=list(map(int,input("Enter partition sizes: ").split()))  
procs=list(map(int,input("Enter process sizes: ").split()))  
used=[0]*len(parts)  
  
for i,p in enumerate(procs):  
    idx=None  
    for j in range(len(parts)):  
        if not used[j] and parts[j]>=p:  
            idx=j; used[j]=1; break  
    if idx is not None:  
        print(f"P{i+1}({p}) → Partition{idx+1} (Internal Frag={parts[idx]-p})")  
    else:  
        print(f"P{i+1}({p}) → Not Allocated")
```

```
(kashvi@LAPTOP-2SJNMAE1)-[~]  
$ python3 task3_mft.py  
Enter partition sizes: 100 500 200 300 600  
Enter process sizes: 212 417 112 426  
P1(212) → Partition2 (Internal Frag=288)  
P2(417) → Partition5 (Internal Frag=183)  
P3(112) → Partition3 (Internal Frag=88)  
P4(426) → Not Allocated
```

→ MVT (variable partition)

```
(kashvi@LAPTOP-2SJNMAE1)-[~]  
$ nano task3_mvt.py
```

```
kashvi@LAPTOP-2SJNMAE1: ~ - x + v
GNU nano 8.1 task3_mvt.py *
mem=int(input("Enter total memory size: "))
free=[(0,mem)]
allocs={}

while True:
    cmd=input("cmd (alloc/free/state/exit): ").split()
    if not cmd: continue
    if cmd[0]=='alloc':
        name,sz=cmd[1],int(cmd[2])
        for i,(s,l) in enumerate(free):
            if l>=sz:
                allocs[name]=(s,sz)
                del free[i]
                if l>sz: free.insert(i,(s+sz,l-sz))
                print(f"{name} allocated at {s}")
                break
            else: print("Not enough space")
    elif cmd[0]=='free':
        if cmd[1] in allocs:
            s,sz=allocs.pop(cmd[1])
            free.append((s,sz))
            free=sorted(free,key=lambda x:x[0])
            merged=[]
            for a,b in free:
                if not merged: merged.append((a,b))
                else:
                    pa,pb=merged[-1]
                    if pa+pb==a: merged[-1]=(pa,pb+b)
                    else: merged.append((a,b))
            free=merged
            print(f"{cmd[1]} freed")
        else: print("Process not found")
    elif cmd[0]=='state':
        print("Allocated:",allocs)
        print("Free:",free)
    elif cmd[0]=='exit': break
```

```
(kashvi@LAPTOP-2SJNMAE1)-[~]
$ python3 task3_mvt.py
Enter total memory size: 1000
cmd (alloc/free/state/exit): alloc P1 200
P1 allocated at 0
cmd (alloc/free/state/exit): alloc P2 300
P2 allocated at 200
cmd (alloc/free/state/exit): state
Allocated: {'P1': (0, 200), 'P2': (200, 300)}
Free: [(500, 500)]
cmd (alloc/free/state/exit): free P1
P1 freed
cmd (alloc/free/state/exit): state
Allocated: {'P2': (200, 300)}
Free: [(0, 200), (500, 500)]
cmd (alloc/free/state/exit): exit
```